

# Redes de Computadoras

## Obligatorio 1

Facultad de Ingeniería  
Instituto de Computación  
Departamento de Arquitectura de Sistemas

### Nota previa - IMPORTANTE

Se debe cumplir íntegramente el “Reglamento del Instituto de Computación ante Instancias de No Individualidad en los Laboratorios”, disponible en el EVA.

En particular está prohibido utilizar soluciones, código o documentación de otros estudiantes, de otros años, de cualquier índole, o hacer público código o soluciones a través de cualquier medio (EVA, correo, papeles sobre la mesa, etc.).

## Introducción

### Forma de entrega

Una clara, concisa y descriptiva documentación es clave para comprender el trabajo realizado. La entrega de la tarea consiste en un único archivo obligatorio1GrupoGG.tar.gz que deberá contener los siguientes archivos:

- Un documento llamado Obligatorio1GrupoGG.pdf donde se documente todo lo solicitado en la tarea. GG es el número del grupo. La documentación deberá describir claramente su solución, las decisiones tomadas, los problemas encontrados y posibles mejoras, y las pruebas realizadas.
- El código fuente del programa (**en lenguaje Python**) e instrucciones claras de cómo ejecutar el sistema.

La entrega se realizará en el sitio del curso, en la plataforma EVA.

### Fecha de entrega

Los trabajos deberán ser entregados **antes del viernes 11/09/2026 a las 23:59 horas**. No se aceptará ningún trabajo pasada la citada fecha y hora. En particular, no se aceptarán trabajos enviados por e-mail a los docentes del curso.

### Uso de herramientas de Inteligencia Artificial

En este obligatorio se permite el uso moderado de herramientas de IA para comprender el enunciado, aclarar conceptos, revisar soluciones propias, detectar errores y recibir ayuda puntual en la implementación de partes específicas.

No está permitido proporcionar el enunciado completo a una herramienta de IA para obtener la solución total o una parte sustancial del código, ni utilizarla para generar el análisis o el texto completo del informe.

Todo uso de IA deberá declararse en la entrega, indicando la herramienta utilizada y describiendo brevemente para qué se empleó. Los estudiantes deberán comprender, revisar y poder justificar todo el contenido entregado. Se hará especial énfasis en esto durante la defensa del trabajo.

## Objetivo del Trabajo

- Familiarizarse con conceptos básicos sobre redes e Internet y manejar herramientas para diagnóstico y *debug* de la red.
- Aplicar los conceptos teóricos de capas de aplicación y transporte, la utilización de la API de sockets de TCP y de UDP, y la arquitectura de aplicaciones cliente-servidor.

## Descripción general del problema

Se desea implementar un sistema básico de monitorización de algunos parámetros que permiten conocer de manera primaria el uso de los recursos de las computadoras.

La arquitectura cliente-servidor implica la configuración de 2 tipos de clientes (*comun* y *admin*) y de un único servidor que interactúa con aquellos intercambiando información, tanto sobre UDP como sobre TCP.

## Entorno de trabajo

Se podrá realizar la tarea en el entorno de trabajo que considere mas cómodo y adecuado, por ejemplo su computadora personal o las PCUnix de la Facultad.

Sin embargo, debe tener en cuenta que:

- Para resolver la **Parte 1** puede utilizar las herramientas disponibles en su máquina.
- Para la **Parte 2** necesitará tener instalado Python 3.5 o superior y también se recomienda que utilice un ambiente Linux para evitar problemas con la API de sockets. Las máquinas de las salas PCUnix cumplen con este requisito.

## Parte 1

El objetivo de esta primera parte es conocer y manejar las herramientas que en el caso de este obligatorio resultan útiles para resolver la Parte 2.

### **Análisis de paquetes y protocolos**

1. Investigue las herramientas `tcpdump` [1] y `wireshark` [2]. Cree un reporte técnico muy breve (una carilla, máximo dos) mencionando sus principales características y utilidad. Debe concluir si son equivalentes, en que sistemas operativos funcionan, si permiten capturar paquetes en vivo, si se pueden guardar para ser analizados en otro momento, si se puede elegir qué paquetes capturar (acá pueden existir muchos criterios de filtro), si es posible seleccionar en que interfaces de red capturar tráfico, si son interoperables entre ellas, entre otras.

*Nota: en este contexto, el concepto “captura” refiere a los archivos con formato pcap que se puedan generar. No refiere, por ejemplo, a imágenes a incluir en los informes, lo cual desaconsejamos fuertemente realizar.*

2. Ejecute `tcpdump` con las opciones necesarias para capturar tráfico únicamente en la interfaz de loopback, IP `127.0.0.1` y el puerto `2026`, y guarde el resultado en un archivo `servidor.pcap` (archivo que deberá entregar), para su análisis posterior. En otra terminal ejecute el archivo `servidor.py` [3]. En cada una de dos nuevas terminales ejecute el comando `telnet 127.0.0.1 2026` para abrir dos sesiones con el servicio.

En las sesiones abiertas con `telnet`, puede enviar dos tipos de mensajes:

- `par un_número` – el servidor devuelve un número par repetido `un_número` cantidad de veces
- `impar un_número` – el servidor devuelve un número impar repetido `un_número` cantidad de veces

La cantidad de sesiones simultáneas (terminales) debe ser al menos 2, y se sugiere que envíe los mismos mensajes, y más de una vez, por las sesiones abiertas.

Interrumpa la captura y abra en `wireshark` el archivo `servidor.pcap`.

Si todas las sesiones `telnet` son establecidas al mismo servidor y al mismo puerto, ¿es posible que los mensajes de una sesión se vean en las otras? Analice cómo identificar a partir de la información contenida en la captura a qué sesión pertenece cada mensaje enviado.

3. *Sugerencia: reducir al mínimo posible la cantidad de pestañas o ventanas del navegador abiertas, así como otras aplicaciones que no sean estrictamente necesarias, para así evitar “ensuciar” con otro tráfico no relevante las capturas a realizar y analizar.*

Capture tráfico (guardarlo en un archivo denominado `https_web.pcap`, el que deberá entregar) mientras accede desde un navegador a la URL <https://www.fing.edu.uy>, y luego analice la captura con `wireshark`. Agregue las columnas de puerto de origen y puerto de destino a la visualización (si no están visibles) y luego cree un “Display

Filter” para quedarse solo con los paquetes que contienen la dirección IP del servidor y el puerto 443. Realice el mismo análisis de la nueva captura que en el punto anterior. Si la especificación del protocolo HTTP [4] dice que los mensajes son compuestos por texto, ¿por qué no puede ver su contenido?

Repita el experimento pero ahora accediendo desde el navegador a la URL <http://www.neverssl.com>, guardando la captura en un archivo denominado `http_web.pcap`, el que también deberá entregar.

Explique qué cambia respecto al caso del sitio web de Fing en relación a qué mensajes y contenidos se pueden visualizar.

Identifique los encabezados `Content-Type` y `Content-Length` en la respuesta al pedido, explique el significado general de ambos y justifique el valor que tienen en la captura.

## Comando *ping*

Investigue el principio de funcionamiento de la utilidad `ping` [5].

Debe ser capaz de entender la finalidad de su uso, el protocolo utilizado, descripción de encabezados, qué mensajes se envían y cuáles se reciben. Para esto se recomienda hacer capturas de ejecuciones del comando a diferentes sitios e identificar en las capturas todo lo que resultó de su estudio.

Finalmente en base a lo anterior debería ser capaz de entender como se calculan los resultados que el comando muestra en pantalla.

No se requiere documentar en el informe a entregar nada relativo a esta parte.

## Comando *traceroute*

Investigue el principio de funcionamiento del comando `traceroute` [6] (equivalente en Windows: `tracert` [7]).

Debe ser capaz de entender la finalidad de uso, el análisis de los dos protocolos que pueden ser usados por esas herramientas, descripción de encabezados, qué mensajes se envían y cuáles se reciben.

Es fundamental que entienda con exactitud la forma en que la herramienta determina que el destino fue alcanzado (como se traduce eso a mensajes de los protocolos involucrados). Para esto se recomienda hacer capturas de ejecuciones del comando a diferentes sitios e identificar en ellas todo lo que resultó de su estudio.

Finalmente en base a lo anterior debería ser capaz de entender cómo se calculan los resultados que el comando muestra en pantalla.

No se requiere documentar en el informe a entregar nada relativo a esta parte.

## Reflexión

Un usuario informa que no puede acceder a un sitio web, que el “ping a la dirección IP del servidor web falla”, y que un `traceroute` dirigido a la misma dirección IP muestra que los paquetes llegan hasta la red del proveedor de *hosting* del servidor, “y no siguen”.

¿Qué hipótesis podrías plantear y qué otras herramientas o pruebas utilizarías antes de concluir (o no) que el sitio web está fuera de servicio?

## Parte 2

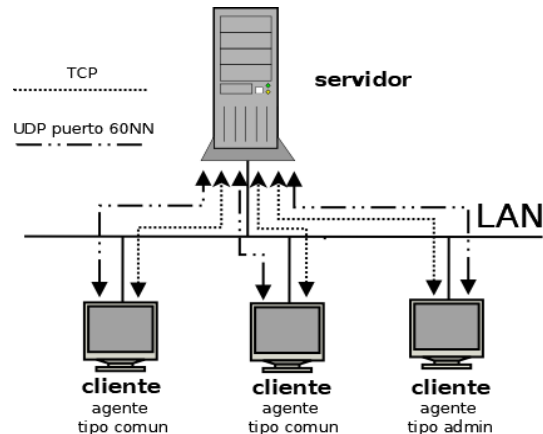
### Descripción general del sistema de monitorización

Se desea implementar un sistema para monitorizar el estado de distintas computadoras (“clientes”) dentro de una red local. Cada computadora ejecuta un **agente** y existe un único **servidor** encargado de recibir la información de monitorización y registrar el estado de los agentes.

Se implementarán dos tipos de agentes: tipo *comun*, que envía al servidor determinada información, y tipo *admin*, que puede consultarla.

Cada cliente, o es un agente *comun* o es un agente *admin*.

La función de los agentes tipo *comun* es transmitir valores de determinadas métricas al servidor y eventuales alertas asociadas, y responder a solicitudes de éste.



Los agentes tipo *admin* podrán conectarse al servidor para consultar qué agentes *comunes* se encuentran conectados actualmente y obtener algunos valores recibidos de los mismos, así como también los procesos en ejecución actualmente.

El servidor sólo interactúa con los agentes (*comun* y *admin*), en un modelo de topología lógica “estrella”.

Los agentes no interactúan directamente entre sí. Si un agente *admin* requiere determinada información de un agente *comun*, la solicitud siempre se la hará al servidor y éste al agente *comun*.

#### Fase de descubrimiento

Cada agente (*comun* o *admin*), al ser iniciado, envía un mensaje broadcast IP al puerto UDP 60NN, donde NN es el número de grupo; el objetivo es determinar en qué dirección IP se encuentra el servidor de monitorización, y en qué puerto TCP está escuchando el servicio de monitorización.

El servidor responde al agente indicando el puerto TCP en el que acepta conexiones para,

- en el caso del agente *comun*, enviar valores de métricas y otra información que le solicite el servidor.
- en el caso del agente *admin*, intercambiar información relacionada con los agentes *comunes*.

Además, indica los valores umbrales a considerar de las métricas monitorizadas de forma de generar alertas (ver mas adelante).

#### Fase de monitorización

Luego que cada agente *comun* se conecta al servidor (al puerto TCP informado durante la fase de descubrimiento), éste le asigna un identificador y el agente comienza a enviar periódicamente (cada 15 segundos) los valores actuales de determinadas métricas de monitorización.

Las métricas monitorizadas por el servidor en cada agente *comun* son:

- CPU (porcentaje de uso general del procesador)
- MEM (porcentaje de memoria ocupada)

El servidor debe almacenar para cada agente *comun* registrado, los últimos 10 valores recibidos de cada métrica (CPU y MEM).

Además, el agente *comun* notificará al servidor en caso de que alguno de los valores monitorizados supere los valores umbral definidos e informados por el servidor al momento del registro. Esta información se registrará en una bitácora (*log*) del servidor.

Por otro lado, el servidor podrá solicitar a un agente *comun* el detalle de los procesos ejecutando en él; motivado ello en una solicitud recibida al respecto desde un agente *admin*.

Los agentes *admin* podrán solicitar al servidor:

- el listado de agentes *comunes* conectados al servidor.

Para ello, el usuario desde la línea de comandos del agente *admin* deberá escribir el comando L, lo que se traducirá en el mensaje de red LIST\_AGENTS (ver más adelante). En respuesta a ello se desplegará la información recibida.

- los últimos valores disponibles de una métrica específica para un agente *comun* determinado.

Para ello, el usuario desde la línea de comandos del agente *admin* deberá escribir el comando M <x> <metric>, donde x es un número ordinal que asocia al agente *comun* seleccionado de la lista de agentes recibida y metric es la métrica a consultar (CPU o MEM), lo que se traducirá en el mensaje GET\_METRIC correspondiente (ver más adelante). En respuesta a ello se desplegará la información recibida.

- el detalle de los procesos que se está ejecutando en un agente *comun* determinado.

Para ello, el usuario desde la línea de comandos del agente *admin* deberá escribir el comando P <x>, donde x es un número ordinal que asocia al agente *comun* seleccionado de la lista de agentes recibida, lo que se traducirá en el mensaje GET\_PROC correspondiente (ver más adelante). En respuesta a ello se desplegará la información recibida.

## Protocolo a implementar

El protocolo de comunicación entre los agentes y el servidor debe poder intercambiar los siguientes mensajes:

Sobre UDP,

- DISCOVER

Mensaje enviado por un agente para descubrir al servidor.

- SERVER <umbral\_cpu> <umbral\_mem> <puerto\_tcp>

Mensaje enviado por el servidor en respuesta al mensaje DISCOVER, indicando los valores umbrales a considerar para las métricas CPU y MEM, y el puerto TCP en el que acepta conexiones de los agentes.

Sobre TCP, para el caso de un agente *comun*:

- REGISTER <clave>

Mensaje enviado por un agente *comun* para registrarse en el servidor donde <clave> es una frase en texto plano secreta solo conocida por agentes y servidor.

- REG\_RESP

Mensaje enviado por el servidor como confirmación al mensaje REGISTER de un agente *comun*.

- METRIC <nombre\_metrica> <valor>

Mensaje enviado por un agente *comun* para informar el <valor> de la métrica <nombre\_metrica>.

- GET\_PROC

Mensaje enviado por el servidor a un agente *comun* solicitando que informe los procesos corriendo en el equipo y sus PIDs (*Process IDs*).

- PROC <pid>:<name>, .. , <pid>:<name>

Mensaje enviado por el agente *comun* en respuesta al mensaje GET\_PROC.

- ALERT <nombre\_metrica> <valor>

Mensaje enviado por un agente *comun* para informar al servidor que el agente tiene el <valor> de la métrica <nombre\_metrica> que excede el umbral definido.

- ERROR

Respuesta enviada cuando el mensaje recibido es inválido o incorrecto.

- END

Mensaje enviado por un agente *comun* eliminar su registro del servidor y cerrar la conexión.

Sobre TCP, para el caso de un agente *admin*:

- ADMIN <clave>  
Mensaje enviado por un agente *admin* al iniciar su conexión con el servidor donde <clave> es una frase en texto plano secreta solo conocida por agentes y servidor.
- ADMIN\_RESP  
Mensaje enviado por el servidor como confirmación al mensaje ADMIN, de un agente *admin*.
- LIST\_AGENTS  
Mensaje enviado por un agente *admin* al servidor para obtener el listado de identificadores de agentes *comunes* actualmente conectados al servidor.
- AGENTS <cantidad> <id\_agente\_1> <id\_agente\_2> ... <id\_agente\_n>  
Mensaje enviado por el servidor en respuesta al mensaje LIST\_AGENTS.
- GET\_PROC <id\_agente>  
Mensaje enviado por un agente *admin* al servidor, solicitando que informe los procesos corriendo en el agente *comun* <id\_agente> y sus PIDs (*Process IDs*).
- PROC <id\_agente> <pid>:<name>, .. , <pid>:<name>  
Mensaje enviado por el servidor en respuesta al mensaje GET\_PROC.
- GET\_METRIC <id\_agente> <nombre\_metrica>  
Mensaje enviado por un agente *admin* para obtener desde el servidor los últimos valores de la métrica <nombre\_metrica> del agente *comun* <id\_agente>.
- MEASUREMENTS <id\_agente> <nombre\_metrica> <cantidad\_de\_valores> <valor\_1> <valor\_2> ... <valor\_n>  
Mensaje enviado por el servidor en respuesta al mensaje GET\_METRIC.
- ERROR  
Respuesta enviada cuando el mensaje recibido es inválido o incorrecto.
- END  
Mensaje enviado por un agente *admin* para cerrar la conexión.

Otras consideraciones de los mensajes del protocolo:

- Todos los mensajes son de texto y terminan con \n.
- Sólo el mensaje DISCOVER es enviado a la dirección de *broadcast* IP. Todos los restantes mensajes son *unicast*.



## Consideraciones generales

En caso de recibir el servidor algún mensaje desde un agente *comun* no registrado actualmente (intercambio adecuado de los mensajes REGISTER y REG\_RESP), el servidor debe responder con un mensaje ERROR.

En caso de recibir el servidor algún mensaje desde un agente *admin* no registrado actualmente (intercambio adecuado de los mensajes ADMIN y ADMIN\_RESP), el servidor debe responder con un mensaje ERROR.

Para la obtención de los valores requeridos del sistema (CPU, MEM y PROC) se puede utilizar el módulo *psutil* (*process and system utilities*) [8] de Python.

Debe implementar control de errores donde corresponda y la posibilidad de reintentos y cierre ordenado de las aplicaciones en caso de no respuesta de alguno de los mensajes intercambiados.

No se acepta el uso de bibliotecas que oculten el uso de sockets; debe usar la API de sockets directamente.

## Resumen

### Cliente

Se implementarán dos clientes: *cliente\_comun.py* y *cliente\_admin.py*. Los clientes podrán tener un archivo incluido común para no repetir código.

- *cliente\_comun.py* implementará todo lo indicado en la letra para el caso de un agente tipo *comun*.
- *cliente\_admin.py* que implementará todo lo indicado en la letra para el caso de un agente tipo *admin*.

### Servidor

El servidor *servidor.py*, deberá implementar el servicio según se describe en la letra.

### En resumen, se pide que:

- a) Implemente en lenguaje Python, los 2 clientes y el servidor solicitados.
- b) Pruebe las implementaciones realizadas, ejecutando más de una instancia de ambos tipos de agentes. El desarrollo se puede realizar y probar en una única máquina. También es posible probarlo en el laboratorio de Fing entre varias computadoras, cada una ejecutando un cliente.

## Referencias y Bibliografía Recomendada

- [1] <https://www.tcpdump.org/manpages/tcpdump.1.html>. Última visita: Agosto 2026
- [2] Analizador de Tráfico Wireshark. Accesible en línea: <http://www.wireshark.org/>. Última visita: Agosto 2026
- [3] <https://eva.fing.edu.uy/mod/resource/view.php?id=84817>. Última visita: Agosto 2026
- [4] <https://datatracker.ietf.org/doc/html/rfc2616> Última visita: Agosto 2026
- [5] ping(8) - Linux man page. Accesible en línea: <https://linux.die.net/man/8/ping> Última visita: Agosto 2026.
- [6] traceroute(8) - Linux man page. Accesible en línea: <https://linux.die.net/man/8/traceroute> Última visita: Agosto 2026.
- [7] tracert Documentation. Accesible en línea: <https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/tracert> Última visita: Agosto 2026.
- [8] psutil documentation. Accesible en línea: <https://psutil.readthedocs.io/stable/> Última visita: Agosto 2026.